# Test Plan Documentation

## Interactive House – SG2 Units


Revision History....1

Test Plan....3

1.1 General....3

1.2 Mobile....3

1.2.1 Scope....3

1.2.2 Testing Approach & Tools....3

1.2.3 Verification & Validation (V&V)....5

1.2.4 Test Examples....5

Figures....5

Mobile....6

Figure 1: Mobile Testing Structure in Visual Studio Code....6

Figure 2: Mobile Total Tests....6

Figure 3: Mobile Coverage....7

# Revision History

| Date | Version | Description | Author |
|---|---|---|---|
| 15/Mar/26 | 1.0 | Initial Test Plan for React Native/Expo Using Jest (1. General and 2. Mobile) | Daniel Jönsson & Ali Daoud |
| 15/Apr/26 | 1.1 | Add UI/Unit Tests to Mobile App and Coverage. Add Table of Contents and New Figures (1, 2, 3) | Daniel Jönsson |
| | | | |
| | | | |

# Test Plan

## 1.1 General

In our role as the Units subgroup, we are responsible for the mobile app and web interface that communicates with the server and database. We will test the data exchange between our unit and the database to ensure commands are sent accurately. In line with SDG 3 and SDG 10, we will test the interface to ensure it supports users with high grades of disabilities, emphasizing self-control and independence. The tests are also made to save time, so we do not have to debug everything ourselves.

## 1.2 Mobile

### 1.2.1 Scope

Our testing focuses on ensuring that our React Native application can successfully: Receive and render dynamic User Interfaces (UIs) distributed by the server and database from the observed device states in the Arduino Smart Home, such as data from the temperature sensor or control house devices, such as toggling the light "ON" or "OFF" through the database. We will also verify that the UI adapts correctly to the mobile phone's touch screen or alternative inputs like speech recognition.

### 1.2.2 Testing Approach & Tools

We will use the “Jest Framework” to perform both Unit and UI Testing on individual React Native components and logic functions. We will also use the Expo Go environment to manually validate that the touch-screen interactions (buttons, gestures) work as intended on mobile hardware.

All tests run locally using: “npm test”

The test setup is configured using Jest with Expo support. The following development dependencies are used for testing: jest, jest-expo, @testing-library/react-native,
react-test-renderer, @types/jest.

jest.setup.ts is used for environment setup and required mocks. Test files located in the __tests__ directory. The testing environment is isolated from production configuration files.

### 1.2.3 Verification & Validation (V&V)

For the verification part, we will use the Jest UI/Unit Test Framework to verify that our internal logic meets the requirements defined in our Requirement Document.

For the validation part, we will assess if the final mobile interface successfully supports users with high grades of disabilities.

### 1.2.4 Test Examples

**Authentication and Access Control**
Because our project focuses on a smart home control center context, ensuring only authorized users can access the system is a primary requirement.

We test that the signInWithEmailAndPassword function correctly identifies valid users and provides specific error codes for invalid attempts.

To support the anytime and everywhere nature of ubiquitous computing, we implemented a withTimeout test. This verifies that the app provides immediate feedback if the house server is unreachable, maintaining the user's sense of self-control and independence.

We verify onAuthStateChanged to ensure the app accurately tracks whether a user is currently logged in.

**Database Observation and Device Control**
A major purpose of this project is to study how to communicate with and observe house devices. Our Firestore tests in mobile validate this interaction.

We use Jest here to mock onSnapshot listeners. This ensures that the mobile unit correctly reflects the status of devices as they change in the database, allowing the user to observe the house remotely.

We also test the control aspect of the project by verifying that toggling a switch in our React Native UI sends the correct update command like turning a light "off" to the database. For the error handling, we test how the app reacts to missing device

configurations or database permission errors.

# Figures

## Mobile

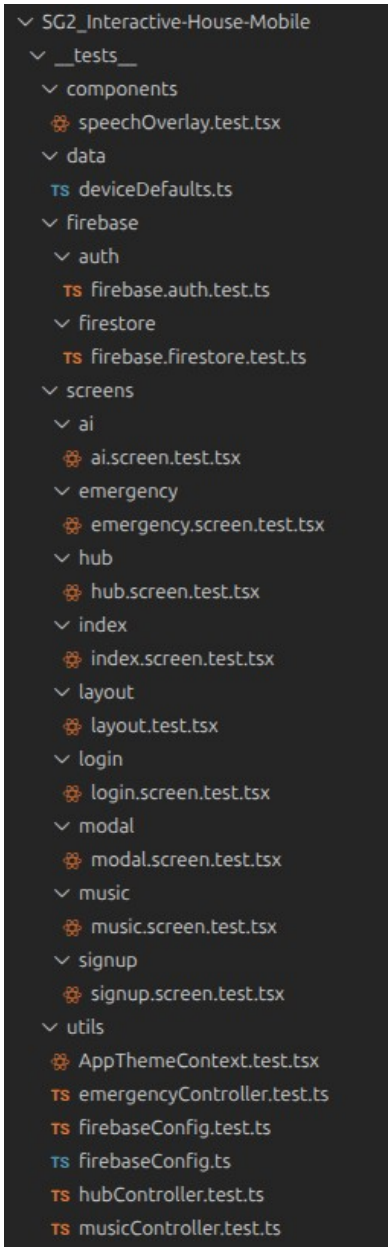


*Figure 1: Mobile Testing Structure in Visual Studio Code*

```
> interactive-house-mobile@1.0.0 test
> jest

 PASS  __tests__/firebase/auth/firebase.auth.test.ts (6.215 s)
 PASS  __tests__/firebase/firestore/firebase.firestore.test.ts (6.573 s)
 PASS  __tests__/screens/layout/layout.test.tsx
 PASS  __tests__/utils/hubController.test.ts
 PASS  __tests__/utils/musicController.test.ts
 PASS  __tests__/components/speechOverlay.test.tsx (8.745 s)
 PASS  __tests__/utils/emergencyController.test.ts
 PASS  __tests__/screens/index/index.screen.test.tsx (9.038 s)
 PASS  __tests__/utils/firebaseConfig.test.ts
 PASS  __tests__/screens/modal/modal.screen.test.tsx (11.514 s)
 PASS  __tests__/screens/music/music.screen.test.tsx (11.696 s)
 PASS  __tests__/utils/AppThemeContext.test.tsx (11.697 s)
 PASS  __tests__/screens/emergency/emergency.screen.test.tsx (6.116 s)
 PASS  __tests__/screens/signup/signup.screen.test.tsx (12.667 s)
 PASS  __tests__/screens/login/login.screen.test.tsx (12.714 s)
 PASS  __tests__/screens/ai/ai.screen.test.tsx (13.627 s)
 PASS  __tests__/screens/hub/hub.screen.test.tsx (14.706 s)

Test Suites: 17 passed, 17 total
Tests:       178 passed, 178 total
Snapshots:   0 total
Time:        17.579 s
Ran all test suites.
```

*Figure 2: Mobile Total Tests*

**All files**

**67.14%** Statements 789/1175 **61.19%** Branches 555/907 **70.23%** Functions 177/252 **68.82%** Lines 764/1110

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

| File | | Statements | | Branches | | Functions | | Lines | |
|---|---|---|---|---|---|---|---|---|---|
| __tests__/data | | 0% | 0/0 | 0% | 0/0 | 0% | 0/0 | 0% | 0/0 |
| app | | 87.6% | 106/121 | 84.33% | 70/83 | 82.5% | 33/40 | 88.79% | 103/116 |
| app/(auth) | | 91.6% | 120/131 | 85.05% | 74/87 | 78.57% | 22/28 | 94.44% | 119/126 |
| app/(tabs) | | 54.81% | 427/779 | 50.07% | 318/635 | 61.43% | 94/153 | 56.59% | 416/735 |
| components | | 93.18% | 82/88 | 92.04% | 81/88 | 100% | 8/8 | 93.9% | 77/82 |
| data | | 100% | 1/1 | 100% | 0/0 | 100% | 0/0 | 100% | 1/1 |
| utils | | 96.36% | 53/55 | 85.71% | 12/14 | 86.95% | 20/23 | 96% | 48/50 |

*Figure 3: Mobile Coverage*

**Statements** are the individual instructions that make a computer perform an action. A single line of code might contain one statement, or it could contain several.

**Branches** refer to the decision points in the code anywhere the control flow can branch off in different directions. This usually involves if/else statements, switch cases, or ternary operators (? :).

**Functions** is the simplest metric. It tracks whether each function or method defined in the code was called at least once during the tests.

**Lines** measure the physical lines of code in your source file that were executed.